

模块七: 智能体开发实战

第二十七讲: 浏览器里的智能体——打造身边的智能体助理

[解释]

本讲概览

本讲的核心任务是把前两讲在 Python 终端里跑通的智能体，搬进浏览器。技术载体是 Chrome 插件（Manifest V3）。

关键设计决策是：不重写 AI 逻辑。Python 后端（FastAPI）继续负责大模型推理，Chrome 插件只负责两件事：从网页上提取文本（感知），以及把后端返回的结果渲染出来（呈现）。这就是 C/S 前后端分离架构的实际应用。

本节目标: 将智能体搬进浏览器

本节课的核心知识流线：

1. **[理解层]** 了解 Chrome 插件 (Manifest V3) 的三个核心文件：`manifest.json`、`content.js`、`popup.js`，理解各自的职责。
2. **[应用层]** 用 JavaScript 读取网页 DOM，提取页面上的目标文本。
3. **[分析层]** 通过前端 `fetch` 请求将数据发送至 L26 的 Python 后端，并验证大模型返回结果能否正确渲染到页面上。

范式目标：网页智能体基座 (Web Agent Skeleton)

本节课以“网页内容提取与大模型分析”为具体练习，但核心目的是掌握一套通用开发范式。

一旦学会从页面 DOM 中提取内容并发送给云端大模型处理，同样的技术可以改造为：网页长文一键总结、自动提取关键数据填表、自动爬取文献等浏览器辅助工具。

[解释]

教学目标解读

本页列出了三个层次的目标。理解层要求知道 Chrome 插件的三个核心文件各自负责什么；应用层要求能写 JS 代码从网页中提取文本；分析层要求能把前端 `fetch` 请求和后端 API 串通，验证整条链路。右侧的范式目标提示框点明了一个重要观点：本讲表面上在做网页文本提取，但掌握的是一套通用的开发范式——提取 DOM + 桥接云端大模型。同样的技术可以改造为网页总结、自动填表、文献爬取等工具。

一、痛点分析

为什么要开发 Chrome 插件？

[解释]

章节定位：需求分析

本章分析传统 AI 使用方式的体验问题（人找工具 vs 工具找人），为开发 Chrome 插件提供动机。

从“人找工具”到“工具找人”

在传统的 AI 脚本开发中，用户体验是非常割裂的：

- **传统流程：**用户在查阅网页文献资料 -> 需要提取信息 -> 手动复制 -> 切换到大模型网页或终端脚本 -> 粘贴提问 -> 等待结果。

这个流程中，用户被迫离开了当前的“工作语境 (Context)”。

- **插件流程：**用户在查阅网页资料 -> 点击浏览器上的插件图标 -> 插件原地提取网页内容，并直接渲染出大模型的处理结果。

工程产品化思维

真正的 AI 产品，不仅仅是提供一个“聊天框”。优秀的智能体应该能够嵌入用户的高频工作环境（如浏览器、IDE、聊天软件）中，在不需用户切换界面的前提下提供服务。这就是所谓的“环境嵌入式智能”。

[解释]

环境嵌入式智能

本页对比了两种 AI 使用方式。传统方式要求用户离开当前工作环境，手动复制粘贴到大模型界面——这叫“人找工具”，打断了用户的工作流。

而 Chrome 插件可以直接驻留在用户正在浏览的网页上，一键完成提取和分析——这叫“工具找人”。用户不需要切换窗口，AI 能力直接嵌入了日常工作环境。

这个概念在工业界叫做“环境嵌入式智能”。市面上流行的 AI 文献总结插件、一键填表插件，都是这个思路的产品化应用。

二、跨端架构迁移

核心逻辑解耦与服务复用

[解释]

章节定位：架构设计

本章讲解从 Python 终端到浏览器的架构迁移策略：前端换成 JS，后端完全复用 L26 的 FastAPI 服务。

跨越环境的 C/S 协同架构

我们在 L26 中建立的流水线（VLM 读图 -> DeepSeek 推理）是基于 Python 服务端的。

现在到了浏览器环境中，编程语言变成了 JavaScript，但智能体的 **核心架构 (Architecture)** 依然适用：

1. **感知通道变更**：之前是上传图片给 VLM 看；现在智能体直接使用 JS 抓取网页文字（读取 DOM 元素）。
2. **推理侧重复用**：直接复用 L26 跑通的 FastAPI 后端 API 服务！前端只需发起 HTTP 请求。

C/S 前后端分离架构的核心优势

前端 (Client)：Chrome 插件 (JavaScript)。负责在网页上“看”（提取 DOM 内容），并“呈现”（渲染大模型结果）。

后端 (Server)：我们在 L26 搭建的 Python FastAPI。继续负责“想”（大模型推理与业务逻辑加工）。

这种前后端解耦的设计，让我们可以随时更换客户端外壳，而无需重复编写 AI 核心逻辑。

[解释]

C/S 前后端分离的工程价值

本页是理解整节课架构的关键。从 L25/L26 的纯 Python 脚本到 L27 的 Chrome 插件，变化的是前端载体（从终端变成了浏览器），不变的是后端逻辑（FastAPI + 大模型推理）。

具体来说：感知通道从上传图片给 VLM 变成了用 JS 读取网页 DOM 文本；推理环节完全复用 L26 的 Python 后端，前端只需要发一个 HTTP 请求即可。

这种设计的最大好处是：如果将来想把 AI 能力做成手机 App 或微信小程序，只需要再写一个新的前端外壳，后端 API 一行代码都不用改。

三、插件实战

敏捷开发：端侧交互实现

[解释]

章节定位：分步编码

本章分三步完成 Chrome 插件：感知网页内容（DOM 读取）、跨端协同（fetch 后端）、闭环回传（黑客松打卡）。

核心组件: Chrome 插件三剑客

一个标准的 Chrome 插件由三个核心文件组成：

1. **manifest.json**：插件的身份证。
定义了插件的名字、权限（比如允许读取哪些网站）、以及用到的 JS 文件。
2. **content.js**：内容脚本。它会被注入到网页内部，可以自由地读取和修改当前网页的内容（感知器官）。
3. **popup.js** / **popup.html**：弹出层脚本。点击插件图标弹出的那个小窗口，用来与用户交互并发起网络请求。

```
// manifest.json 基础配置
{
  "manifest_version": 3,
  "name": "AI 学伴随身听",
  "version": "1.0",
  "permissions": ["activeTab", "scripting"],
  "action": {
    "default_popup": "popup.html"
  },
  "host_permissions": [
    "http://localhost:8000/*", // 允许请求本地的 Python API
    "https://syb.ncu.edu.cn/*" // 允许请求黑客松判卷服务器
  ]
}
```

[解释]

Chrome 插件的三文件结构

本页介绍了 Chrome 插件的最小组成单元。manifest.json 是必须的配置文件，定义了插件名称、所需权限和引用的脚本文件。content.js 是注入到网页内部的脚本，可以读取和修改页面的 DOM 内容。popup.js 和 popup.html 构成了点击插件图标后弹出的小面板界面。

右侧的 JSON 示例中有几个关键字段：permissions 里的 activeTab 表示插件只能访问当前活跃的标签页；scripting 表示允许注入脚本；host_permissions 里的 <http://localhost:8000/>* 授权插件向本地 Python 服务器发送请求。

Step 1: 感知网页内容 (DOM 读取)

我们要让智能体“看”到黑客松网页上的英文题干。

在 `popup.js` 中，我们通过 Chrome API 注入一段短小的感知脚本，去抓取网页上用户选中的文字，或者指定 ID 的文字：

```
// 在 popup.js 中获取当前网页的内容
async function getWebText() {
  let [tab] = await chrome.tabs.query({ active: true, currentWindow: true });

  let results = await chrome.scripting.executeScript({
    target: { tabId: tab.id },
    func: () => {
      // 感知：读取用户在网页上选中的英文
      return window.getSelection().toString();
    }
  });

  return results[0].result; // 返回抓取到的文本
}
```

从物理视觉到原生数字解析

在 L25 中，我们依赖 VLM 的视觉理解能力才能看懂一张图片上的字。而在浏览器这种原生数字环境中，我们仅仅用一行

`getSelection()` 就完成了高精度的内容提取。善用环境原生接口，是工程效率的关键。

[解释]

DOM 读取的代码逻辑

本页的代码展示了插件如何从网页上提取文本。流程分三步：(1) `chrome.tabs.query` 获取当前活跃标签页的 ID；(2) `chrome.scripting.executeScript` 向该标签页注入一段匿名函数；(3) 注入的函数调用 `window.getSelection().toString()` 获取用户选中的文字并返回。

右侧的 insight 提示框对比了 L25 的 VLM 视觉识别和本讲的 DOM 读取：在浏览器环境中，文字本身就是结构化数据，不需要消耗算力去看图识字，一行 JS 就能精确提取。

Step 2: 跨端协同与结果渲染

拿到提取的内容后，我们在 `popup.js` 中使用原生的 `fetch` 向我们自己的 Python 后端发起大模型推理请求：

```
// 1. 调用本地 Python 的推理接口进行文本分析/加工
let analyzeRes = await fetch("http://localhost:8000/api/analyze", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({ source_text: webText })
});
let analyzeData = await analyzeRes.json();
// 2. 在前端渲染大模型返回的结构化结果
document.getElementById("output").innerText = analyzeData.result_desc;
```

无缝的前后端协作

在这个流程中，浏览器端的 JavaScript（客户端）非常轻量。繁重的多模型调用与 API Key 管理，全部交给了 L26 写好的 Python 后端（服务端）。这就叫**敏捷开发 (Agile Development)**：最大化复用已有资产。

[解释]

fetch 请求与前后端协作

本页的代码展示了前端如何与后端通信。`popup.js` 使用浏览器原生的 `fetch` API 向 <http://localhost:8000/api/analyze> 发送 POST 请求，请求体包含从网页上提取的文本。后端 Python 收到文本后调用大模型处理，再将结果以 JSON 格式返回。前端拿到 JSON 后，用 `document.getElementById` 将结果渲染到弹出面板上。

关键点是：前端 JS 代码非常轻量，不包含任何 AI 逻辑和 API Key。所有繁重的推理工作和密钥管理都在后端完成，和 L25 讲过的全栈安全铁律完全一致。

Step 3: 闭环回传与黑客松打卡

作为智能体全栈开发的最后一环，我们需要将大模型处理好的数据，回传给我们的核心业务系统（黑客松平台）。

在 `popup.js` 拿到结果后，追加一个网络请求，将成果进行上报打卡：

```
// 3. 将结果上报黑客松平台获取积分
await fetch("https://syb.ncu.edu.cn/smart-class/api/hackathon/extension/submit", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "X-API-Key": "YOUR_API_KEY"
  },
  body: JSON.stringify({
    question_id: questionId, // 从靶机页面 DOM 中提取的隐藏 ID
    translation: analyzeData.result_desc
  })
});
alert("翻译提交成功，积分已到账！");
```

数据闭环是最后一步

真实的商业产品中，AI 处理完数据后，一定要落库（保存到数据库）或者驱动下一个业务流。

在这里，我们通过标准的 API 请求，让你的 Chrome 插件直接与远端的黑客松中央服务器完成了联动。

[解释]

数据闭环的业务意义

本页展示了智能体开发的最后一环：将处理结果推送给外部业务系统。代码通过 `fetch` 向黑客松判卷服务器发送 POST 请求，携带从靶机页面提取的 `question_id` 和大模型翻译结果。

这一步的工程意义在于：AI 处理完数据后不能停在原地，必须驱动下一个业务流程（存入数据库、触发通知、更新仪表盘等）。感知、推理、行动三步走完，结果必须落地到真实的业务系统中。这就是数据闭环的概念。

四、小结与课后拓展

工业级产品与作业挑战

[解释]

章节定位：总结与进阶

本章总结架构复用的核心收获，并提出课后进阶挑战：去掉 Python 后端，做纯前端独立插件。

总结：敏捷迭代与架构复用

今天这节课，我们几乎没有写复杂的 AI 提示词，也没有调大模型参数，只是写了几行 JavaScript 和配置文件。

但我们却将一个原本只能在终端里跑的实验脚本，升级成了一个随时随地可用的“浏览器生产力工具”。

这正是软件迭代的敏捷优势：

当我们将核心推理能力封装为独立的后端 API 后，我们可以以较低的成本开发不同的前端外壳（Web 页面、Chrome 插件、甚至是小程序），而核心推理逻辑始终是同一套架构。

课后进阶挑战：剥离后端，打造独立前端智能体 (Option A)

挑战目标：如果想把这个得力助手分享给非技术专业的室友用，他们电脑上根本没有 Python 环境怎么办？

思路设计：直接在 `popup.js` 里写一个设置界面，让用户填入自己的 `SiliconFlow API Key`。然后在 JS 代码中直接向云端发起 `fetch` 请求调用大模型接口（完全跳过本地的 Python FastAPI）。去掉语音播报等后端功能后，插件就变成了一个独立运行的轻量应用，无需任何本地环境。

[解释]

本讲知识体系回顾

本讲的核心收获是架构复用。我们几乎没有写新的 AI 逻辑，只是用几十行 JS 和一个 JSON 配置文件，就把一个终端里的实验脚本升级成了浏览器生产力工具。这得益于 L26 中将推理能力封装为独立 API 的设计决策。

课后挑战（Option A）提出了一个更进阶的思路：完全去掉 Python 后端，让 Chrome 插件直接调用云端大模型 API。这样插件就变成了一个纯前端应用，可以直接分发给没有 Python 环境的用户。但代价是 API Key 暴露在前端，需要让用户自行管理。这是安全性与便捷性之间的典型工程权衡。